

ShelterPulse: Simulation-Based Budget Allocation for Cat Shelters Under Seasonal Uncertainty

true

July 2026

Abstract

Seasonal intake creates a difficult capacity-planning problem for cat shelters: a fixed intervention budget must be allocated before the resulting queues and resource interactions are known. ShelterPulse is an open-source decision-support prototype that combines a SimPy discrete-event model, paired-seed Monte Carlo evaluation, five named baseline allocations, and Gaussian-process Bayesian optimization. In the synthetic Whisker Haven case study, evaluated with 32 replications per allocation, the best named strategy (all budget assigned to adoption events) produced 50.2 mean overflow cat-days (95% CI 25.4–75.0). The best Bayesian-optimization candidate produced 82.1 mean overflow cat-days (95% CI 44.4–119.7), a 90.6% reduction from the equal-allocation baseline but not an improvement over the best named strategy. This negative comparison is retained deliberately: the tool reports all evaluated strategies rather than attributing the best sweep result to the optimizer. The prototype is not calibrated for operational use; its purpose is to make assumptions, uncertainty, resource interactions, and reproducibility explicit.

Introduction

Kitten season creates a recurrent planning problem for shelters. Shelter Animals Count reported that neonates, weaned kittens, and juveniles represented 59% of cat intake across reporting organizations in 2025 [1]. Intake composition and timing vary by organization, but the operational consequence is consistent: housing, isolation, staff,

foster, and adoption capacity interact under uncertain demand.

Managers must decide how to use limited intervention funds before they can observe the realized intake stream. ShelterPulse models four spending categories: foster support, extra clinic hours, temporary isolation capacity, and adoption events. Its primary outcome is **overflow cat-days**, defined here as the time integral of the number of cats queued for general housing. One cat waiting for housing for one day contributes one overflow cat-day.

ShelterPulse is a decision-support prototype, not a calibrated prediction product. Its inputs are synthetic, intervention effects are scenario assumptions, and the model omits important real-world behavior. The intended contribution is a transparent computational workflow:

1. validate a versioned shelter scenario;
2. simulate stochastic intake and lifecycle queues;
3. evaluate allocations with aligned random streams;
4. report uncertainty and named baselines; and
5. retain machine-readable evidence for every published result.

Related Work

Animal-shelter operations research has addressed complementary decisions. Bradley and Rajendran combine length-of-stay prediction with shelter-allocation decisions [2]. Kang and Han formulate operational schedules and space sharing across shelters in the Seoul capital area [3].

Karsten et al. report an observational evaluation of Capacity for Care management across three shelters [4]. ShelterPulse differs in scope: it uses discrete-event simulation to explore within-shelter intervention allocation during a synthetic seasonal surge.

Simulation-based optimization and variance reduction are established methods. Law describes common random numbers (CRN) as a paired-comparison technique [5]. Stout and Goldie and Murphy et al. emphasize that effective CRN requires synchronized random sources, not merely reusing one initial seed [6], [7]. Bayesian optimization uses a probabilistic surrogate and an acquisition function to select expensive black-box evaluations [8], [9]. ShelterPulse applies these techniques as engineering tools; it does not claim methodological novelty.

System Model

Scenario and lifecycle

A Pydantic schema validates the simulation horizon, base seed, intake profile, seasonal events, physical capacity, workforce, foster network, costs, interventions, and intervention budget. Unknown fields are rejected. Scenario values are assumptions, not estimates learned from shelter records.

Each generated cat follows this implemented lifecycle:

1. intake profile assignment;
2. assessment by a vet-tech service resource;
3. optional isolation;
4. medical clearance by a vet-tech service resource;
5. immediate foster placement when the cat is eligible and a foster slot is available, otherwise a request for general housing;
6. an adoption-wait period; and
7. adoption or transfer.

The model does not simulate euthanasia, returns after adoption, adopter choice, or transfers between modeled shelters. Cats still present at the

simulation horizon are counted separately. Conservation is tested as

$$N_{intake} = N_{adopted} + N_{transferred} + N_{remaining}.$$

Resources and workforce approximation

Housing, isolation, and foster capacity are finite **simpy.Resource** pools. A foster placement holds one foster slot until exit and therefore relieves general housing. Eligible cats are neonates, medical/critical cases, or cats arriving when housing is already full; if no foster slot is immediately available they join the normal housing path.

Workforce roles are simplified concurrent service pools. Configured FTE is rounded to a positive worker count for event scheduling, while `hours_per_day` is used to normalize reported utilization. This is not a shift-calendar or labor-scheduling model. Clinic and foster-coordination interventions reduce the corresponding service times according to scenario-configured additional-hours effects. This approximation is a known limitation and is described as such rather than as a replenished daily-hours queue.

Intervention effects and cost semantics

An allocation consists of four non-negative budget shares whose sum cannot exceed one. Scenario-defined effect parameters translate dollars into:

- additional foster slots;
- faster foster coordination;
- faster vet-tech service;
- additional isolation slots; and
- a shorter adoption-wait distribution.

The intervention-spend constraint and simulated operating cost are separate quantities. `is_feasible` means allocated intervention spend is within the configured intervention budget. `mean_total_cost` reports simulated operating cost (fixed, variable, medical, and foster supply costs); it is not compared with the \$5,000 intervention budget.

Simulation Methodology

Intake and service processes

ShelterPulse uses SimPy 4 [10]. Intake is generated as a piecewise non-homogeneous Poisson process by thinning from the maximum configured rate. For candidate time t , an arrival is accepted with probability $\lambda(t)/\lambda_{max}$. Intake times and profiles are generated before lifecycle processes begin.

Assessment and healthy-clearance times use exponential distributions. Medical-hold and isolation-clearance times use gamma distributions, as does adoption waiting for some age classes. These are synthetic modeling assumptions encoded in the engine; they are not empirically calibrated service-time estimates.

Overflow metric

The engine samples the housing queue hourly. If Q_h is the number of cats waiting for housing at hour h , the reported metric is

$$\text{overflow cat-days} = \sum_h \frac{Q_h}{24}.$$

Housing occupancy at capacity with no waiting cat contributes zero overflow. This distinction prevents a full but stable shelter from being mislabeled as having one overflow cat-day for every hour of full occupancy.

Paired-seed comparison

Every allocation in a sweep uses the same replication seed set. Reusing a seed alone is insufficient for CRN because intervention-dependent control flow can change random-number consumption. ShelterPulse therefore separates stochastic sources:

- each seed pre-generates one intake schedule and profile sequence; and
- each cat receives independent streams for assessment, isolation, clearance, adoption wait, transfer, and foster coordination.

This design keeps exogenous intake and corresponding per-cat draws aligned across allocations while allowing intervention effects to change queues and event timing. The current report does not assign a numerical variance-reduction factor; that requires a separate measured comparison against independent streams.

For an allocation evaluated over n replications, the implementation reports sample mean, sample standard deviation, and a two-sided 95% t interval for overflow and operating cost. The intervals describe Monte Carlo uncertainty under the model, not uncertainty about whether the model matches a real shelter.

Optimization

Search and baselines

The search space contains four spending shares. Bayesian-optimization proposals use the full budget and are sampled on the four-share simplex. Five named strategies are always evaluated:

1. equal allocation: (0.25, 0.25, 0.25, 0.25);
2. all foster: (1, 0, 0, 0);
3. all adoption events: (0, 0, 0, 1);
4. domain heuristic: (0.4, 0, 0.2, 0.4); and
5. zero intervention: (0, 0, 0, 0).

The baseline observations warm-start the Gaussian-process path, but they remain labeled baseline results. A baseline can therefore rank above every Bayesian-optimization candidate, as it does in the current case study.

Implemented optimizer paths

When JAX and `jaxbo` are installed, the optimizer fits a Gaussian process with a Matérn-5/2 kernel. After initial/warm observations, each sequential step samples 256 feasible Dirichlet proposals, evaluates Expected Improvement for those proposals, and simulates the best acquisition value. It does not use continuous L-BFGS-B acquisition optimization.

If JAX/`jaxbo` is unavailable, the fallback is deterministic seeded Dirichlet random search. It is

not a scipy GP fallback. Results from the two paths must therefore identify the environment and optional dependencies used.

Case Study: Whisker Haven

Configuration

Whisker Haven is a synthetic 90-day scenario with 35 housing slots, 5 isolation slots, 1.5 vet-tech FTE, 3 animal-care FTE, 0.5 foster-coordinator FTE, 8 foster slots, and a \$5,000 intervention budget. Base intake is 3.8 cats/day. A 42-day seasonal event multiplies intake by 2.5, giving approximately 581 expected arrivals over the complete horizon before stochastic variation.

The synthetic intake profile assigns 59% to neonatal/weaned categories, 18% to juveniles, and 23% to adults. This deliberately kitten-heavy scenario should not be confused with the national 59% figure, which includes juveniles [1].

The recorded development run evaluated five baselines plus 20 Bayesian-optimization candidates, each with seeds 42–73 (32 replications). The complete results and source-file SHA-256 digests are stored in `docs/whitepaper/evidence/whisker-haven.json`.

Results

The best BO candidate reduced mean overflow by approximately 91% relative to equal allocation and 96% relative to zero intervention. It did not beat all-in-events, though its converged allocation (91% adoption events) moved toward the same strategy the best baseline identifies, consistent with the GP learning from baseline warm-start observations. The strongest conclusion supported by this run is therefore that an event-heavy allocation performs well under the scenario assumptions; it is not evidence that BO universally outperforms simple strategies.

The run took 243.2 seconds on the recorded Windows/Python environment. This is one development measurement, not a cross-platform performance guarantee. Release materials should re-

port a threshold only after repeated final-commit benchmarks.

Limitations

Synthetic and uncalibrated inputs. Whisker Haven is not derived from one shelter’s operational records. Intake composition, service distributions, intervention effects, and costs require local calibration before decision use.

Simplified foster decisions. Eligibility and immediate slot availability determine foster placement. Foster-family preferences, placement failures, returns, and time-varying recruitment are omitted.

Simplified workforce. FTE becomes concurrent service capacity rather than a shift calendar. Service-speed effects approximate additional hours, and no overtime, absence, skill substitution, or hiring delay is modeled.

Point-estimate intervention effects. Dollar-to-resource mappings are deterministic scenario parameters. Real intervention response is uncertain and may be nonlinear.

Single objective and one evidence scenario. The optimizer minimizes modeled housing overflow. The current paper contains one synthetic scenario and does not establish generalization, welfare outcomes, or external validity.

Optimizer evidence. In this run, the best baseline beats BO. More candidates may change the result, but post-hoc increases would not be a fair comparison. A predeclared multi-scenario study is required before making aggregate optimizer-performance claims.

Future Work

1. Calibrate inputs and intervention effects from appropriately governed shelter data.
2. Run the predeclared multi-scenario generalization study and retain unfavorable results.
3. Add robust or multi-objective optimization for overflow, operating cost, and staff work-

Source	Allocation (F/C/I/A)	Mean overflow cat-days	95% CI	Mean operating cost
All-events baseline	0 / 0 / 0 / 1.00	50.2	25.4–75.0	\$87.5
Best BO candidate	0.053 / 0.037 / 0.004 / 0.906	82.1	44.4–119.7	\$88.5
Domain baseline	0.40 / 0 / 0.20 / 0.40	314.7	198.1–431.3	\$91.5
All-foster baseline	1.00 / 0 / 0 / 0	530.0	360.9–699.1	\$93.5
Equal baseline	0.25 / 0.25 / 0.25 / 0.25	874.4	646.7–1102.0	\$92.5
Zero intervention	0 / 0 / 0 / 0	2038.3	1718.7–2357.9	\$93.5

load.

4. Replace the workforce approximation with explicit shifts and skill calendars if data supports that complexity.
5. Evaluate durable workflow orchestration for longer sweeps. The current production architecture remains RabbitMQ locally and SQS/Lambda in production; Temporal is not a production dependency.
6. Expand the validated scenario library while keeping synthetic archetypes clearly labeled.

Sensitivity tornado, timeline, Pareto, and what-if views already exist in the current UI and are therefore not listed as unimplemented future work.

Reproducibility Statement

Evidence generation

From a repository checkout with Python 3.12 and uv:

```
uv sync --extra optimize
uv run --extra optimize python scripts/generate_paper_evidence.py \
  --candidates 20 \
  --replications 32 \
  --output docs/whitepaper/evidence/whisker_haven.json
```

The JSON records the base Git revision, source-file digests, platform, seed set, configuration, elapsed time, complete ranking, uncertainty intervals, operating costs, and allocated intervention spend. It must be regenerated after any model, optimizer, scenario, or dependency change and again on the final release commit.

The same sweep can be inspected through the CLI:

```
uv run --extra optimize shelterpulse optimize \
  --scenario scenarios/whisker_haven.yaml \
  --candidates 20 \
  --reps 32 \
  --top 25 \
  --json \
  --quiet
```

`docker compose up --build` starts the UI, API, RabbitMQ, and worker for interactive use. The development test suite is run from the repository environment, not from the runtime-only Docker image:

```
uv run tox -e lint,security,test,e2e
cd ui
npm ci
npm run type-check
npm run lint
npm run build
npm run cy:run
```

The current report is a development artifact. Replace the revision statement with the final submitted commit or tag only after these commands

```
pass on that past revision
```

License and citation

ShelterPulse is released under the Apache License 2.0.

```
@misc{garciamirez2026shelterpulse,
  author = {García Ramírez, Ricardo},
  title = {ShelterPulse: Simulation-Based Budget Allocation},
  year = {2026},
  url = {https://github.com/ricardogr07/shelter-pulse},
  note = {Development technical report; Apache-2.0}
}
```

Acknowledgments

Kiro assisted with architecture exploration, implementation, testing, infrastructure work, and documentation. Outputs were reviewed against repository code and executable evidence; final CI status must be evaluated on the submitted revision.

References

- [1] Shelter Animals Count, “Preparing for kitten season 2026: Looking at analysis from the 2025 annual data report.” Accessed: Jul. 02, 2026. [Online]. Available: <https://www.shelteranimalscount.org/preparing-for-kitten-season-2026>
- [2] J. Bradley and S. Rajendran, “Increasing adoption rates at animal shelters: A two-phase approach to predict length of stay and optimal shelter allocation,” *BMC Veterinary Research*, vol. 17, no. 1, p. 70, 2021, doi: 10.1186/s12917-020-02728-2.
- [3] J. H. Kang and J. Han, “Optimizing the operation of animal shelters to minimize unnecessary euthanasia: A case study in the seoul capital area,” *Sustainability*, vol. 11, no. 23, p. 6702, 2019, doi: 10.3390/su11236702.
- [4] C. L. Karsten, D. C. Wagner, P. H. Kass, and K. F. Hurley, “An observational study of the relationship between capacity for care as an animal shelter management model and cat health, adoption and death in three animal shelters,” *The Veterinary Journal*, vol. 227, pp. 15–22, 2017, doi: 10.1016/j.tvjl.2017.08.003.
- [5] A. M. Law, *Simulation modeling and analysis*, 5th ed. McGraw-Hill Education, 2015.
- [6] N. K. Stout and S. J. Goldie, “Keeping the noise down: Common random numbers for disease simulation modeling,” *Health Care Management Science*, vol. 11, no. 4, pp. 399–406, 2008, doi: 10.1007/s10729-008-9067-6.
- [7] D. R. Murphy, R. W. Klein, L. J. Smolen, T. M. Klein, and S. D. Roberts, “Using common random numbers in health care cost-effectiveness simulation modeling,” *Health Services Research*, vol. 48, no. 4, pp. 1508–1525, 2013, doi: 10.1111/1475-6773.12044.
- [8] J. Snoek, H. Larochelle, and R. P. Adams, “Practical bayesian optimization of machine learning algorithms,” in *Advances in neural information processing systems*, 2012, pp. 2951–2959.
- [9] P. I. Frazier, “A tutorial on bayesian optimization,” *arXiv preprint arXiv:1807.02811*, 2018, doi: 10.48550/arXiv.1807.02811.
- [10] SimPy Development Team, “SimPy: Discrete-event simulation for python.” Accessed: Jul. 02, 2026. [Online]. Available: <https://simpy.readthedocs.io/>